CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

CO2 – DEBUGGING AND OPTIMISATION TASK

SentinelOps SOC Alert-Triage Module

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Course: Design and Analysis of Algorithms

Maximum Marks: 50

Introduction and Problem Understanding

This report solves the uploaded CSA06 CO2 Debugging and Optimisation Task. The scenario describes SentinelOps, a Security Operations Centre that must process incoming alerts, check blocked IP addresses, find all attack-signature occurrences in log lines, and identify the closest pair of login locations. The brief specifies approximately 50,000 alerts and batches of up to 8,000 login events.

The supplied instructions require debugging the existing code rather than replacing it without explanation. For each correction, this report identifies what is wrong, why it is wrong, how it is fixed and how it is verified. It then optimises the corrected module while preserving outputs.

The supplied code contains nine defects, as stated in the assessment. The defects are grouped below by function. All sample outputs are derived from the supplied test data on page 2 of the assessment.

Given Test Data

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log       = "FAILFAIXFAIL"
pattern   = "FAIL"
events    = [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)]
```

Part (a) – Reproduce and Identify the Defects (10 Marks)

A1. Expected output for each function

1. Sorting alerts: sort by severity in descending order, while preserving the original arrival order for equal severities.

Expected position	Alert	Reason
1	A2 – severity 9	Severity 9; arrived before A4
2	A4 – severity 9	Equal severity; stable arrival order
3	A3 – severity 5	Next highest severity
4	A1 – severity 3	Next
5	A5 – severity 1	Lowest severity

Expected sorted list: [("A2",9), ("A4",9), ("A3",5), ("A1",3), ("A5",1)].

2. Blocklist: 10.0.0.9 → True; 10.0.0.5 → True; 8.8.8.8 → False. A True result means the IP is blocked.

3. String matching: in "FAILFAIXFAIL", the pattern "FAIL" occurs at positions 0 and 8 (zero-based indexing).

4. Closest login pair: the closest two coordinates are (2,3) and (3,4), at distance $\sqrt{2} \approx 1.414$ km. These are events at original indices 0 and 5.

A2. Actual behaviour of the original code

Sorting: the loop condition j > 0 prevents comparison with index 0, and the key is placed at alerts[j] rather than alerts[j+1]. The original output is therefore incorrectly ordered and can overwrite/duplicate an item.

Blocklist: the function returns False on the first iteration whether the current IP matches or not. It never searches beyond the first record and never returns True.

String matching: range(n-m) omits the final valid alignment. Also, checking j == m-1 can report a match even if the mismatch occurred at the final character, because the loop breaks with j at m-1.

Closest pair: min_dist starts at 0 and the condition d > min_dist selects the farthest pair, not the closest. The nested loop also compares each event with itself and counts both (i,j) and (j,i).

A3. Nine defects identified

No.	Function / faulty line	Defect	Effect
1	sort_alerts – line 40	j > 0 should be j >= 0	The first element is never compared; insertion may stop too early.
2	sort_alerts – line 43	alerts[j] = key should be alerts[j+1] = key	Key is written to the wrong slot and can overwrite an existing item.
3	is_blocked – line 49	Matching IP returns False	A blocked IP is incorrectly allowed.
4	is_blocked – line 50	Returns False inside the loop after first non-match	Later blocklist entries are never examined.
5	find_signature – line 57	range(n-m) should include n-m	A signature beginning at the final legal position is missed.
6	find_signature – line 61	j == m-1 does not prove full match	A mismatch on the final character can be reported as a false hit.
7	closest_logins – line 67	min_dist initialized to 0	No valid positive distance can be selected as a minimum.
8	closest_logins – line 73	Uses d > min_dist instead of d < min_dist	The algorithm tracks the maximum/ farthest distance.
9	closest_logins – lines 69–70	Loops over all j including j=i and both pair orders	Self-pairs and repeated unordered pairs are examined.

The nine defects above account for the stated defect count in the assignment brief.

Part (b) – Root Causes and Systematic Debugging (12 Marks)

B1. Corrected sorting function and trace

```
def sort_alerts(alerts):
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
```

```
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
```

Why the fix works: j >= 0 permits comparison with the first element. When the loop stops, j is the position immediately before the insertion point, so the key belongs at j+1. The strict comparison < shifts only lower severities; equal severities are not shifted, preserving arrival order.

Pass	Key being inserted	Action	List after pass
Start	—	Original input	A1:3, A2:9, A3:5, A4:9, A5:1
1	A2:9	Shift A1:3 right; insert at index 0	A2:9, A1:3, A3:5, A4:9, A5:1
2	A3:5	Shift A1:3; stop before A2:9	A2:9, A3:5, A1:3, A4:9, A5:1
3	A4:9	No shift past equal A2:9; stable order	A2:9, A4:9, A3:5, A1:3, A5:1
4	A5:1	No shift; already lower than all	A2:9, A4:9, A3:5, A1:3, A5:1

B2. Corrected blocklist function

```
def is_blocked(blocklist, ip):
    for item in blocklist:
        if item == ip:
            return True
    return False
```

The function returns True only after finding a match. It returns False after the entire list has been checked. This also handles an empty blocklist: the loop executes zero times and the final result is False.

Query	Expected result	Reason
10.0.0.9	True	Found at second position
10.0.0.5	True	Found at first position
8.8.8.8	False	Not present
Any IP with empty blocklist	False	No entries can match

B3. Corrected brute-force string matching

```
def find_signature(log, pattern):
    n, m = len(log), len(pattern)
    hits = []
    if m == 0:
        return list(range(n + 1))
    if m > n:
        return hits
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
```

```
            j += 1
        if j == m:
            hits.append(i)
    return hits
```

The valid starting positions are 0 through n−m inclusive, so the range endpoint must be n−m+1. A match is confirmed only when j reaches m, meaning every character matched. The function returns all occurrences, including overlapping matches. The empty-pattern convention used here is to return every boundary position 0 through n.

Alignment start	Text segment	Comparison result	Action
0	FAIL	F=F, A=A, I=I, L=L	Record position 0
1	AILF	First char A ≠ F	Reject
2	ILFA	First char I ≠ F	Reject
3	LFAI	First char L ≠ F	Reject
4	FAIX	F=F, A=A, I=I, X ≠ L	Reject; not a match
5	AIXF	First char A ≠ F	Reject
6	IXFA	First char I ≠ F	Reject
7	XFAI	First char X ≠ F	Reject
8	FAIL	F=F, A=A, I=I, L=L	Record position 8

Correct result: [0, 8]. Boundary checks: a pattern at the very end is found; a pattern longer than the log returns []; an empty pattern follows the documented convention.

B4. Corrected closest-pair function

```
  def closest_logins(events):
    if len(events) < 2:
        return None, None
    min_dist2 = float("inf")
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d2 = dx*dx + dy*dy
            if d2 < min_dist2:
                min_dist2 = d2
                pair = (i, j)
    return pair, math.sqrt(min_dist2)
```

The loops use j=i+1, so each unordered pair is examined exactly once and no event is compared with itself. The minimum squared distance is initialised to infinity and updated only when a smaller squared distance is found. The square root is calculated once, after the best pair has been identified.

Candidate pair (indices)	Coordinates	Squared distance

(0,5)	(2,3) and (3,4)	1 + 1 = 2
(0,3)	(2,3) and (5,1)	9 + 4 = 13
(1,4)	(12,30) and (12,10)	0 + 400 = 400

The minimum is pair (0,5), with distance $\sqrt{2} \approx 1.414$ km. If fewer than two events are supplied, the corrected function returns (None, None), because a pair does not exist.

B5. Boundary-test summary

Test	**Expected corrected behaviour**
Empty alerts	Return an empty list
Equal alert severities	Keep their original arrival order
Empty blocklist	Return False for every IP
IP at first / middle / last position	Return True
IP absent	Return False
Pattern at final legal log position	Include final starting index
Pattern longer than log	Return an empty hit list
Overlapping signatures	Return all valid start positions
Zero or one login event	Return (None, None)

Part (c) – Optimise the Corrected Module (10 Marks)

C1. Alert sorting for 50,000 alerts

Insertion Sort is adaptive and stable. Its time is O(n+I), where I is the number of inversions relative to descending severity order. If alerts are nearly sorted, I is small and insertion sort can be efficient. However, its worst case is $O(n^2)$, which is risky for 50,000 random alerts. Python's built-in sorted() uses stable Timsort, which is adaptive and O(n log n) worst case; it preserves the same required output and is the recommended production choice.

```
def sort_alerts_optimized(alerts):
    # Python sorting is stable: equal severities retain input order.
    return sorted(alerts, key=lambda alert: -alert[1])
```

If the workload is known to be almost sorted, benchmark insertion sort against Timsort. For a production system with a mixed or uncertain workload, stable Timsort is safer.

C2. Repeated blocklist lookups

```
def make_blocklist_index(blocklist):
    return set(blocklist)

def is_blocked_optimized(blocked_ips, ip):
    return ip in blocked_ips
```

Build the set once, then use membership checks. Building costs O(b) expected time for b blocklist entries; each lookup is expected O(1), compared with O(b) for a sequential scan. The set uses additional memory and must be refreshed when the blocklist changes.

C3. Signature scan without redundant comparisons

For repeated signature scans, use Knuth–Morris–Pratt (KMP). KMP preprocesses the pattern into a longest-prefix-suffix (LPS) table. When a mismatch occurs, it reuses the prefix information instead of comparing characters that are already known to match. It finds all occurrences, including overlaps, in O(n+m) time.

```
def build_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps

def find_signature_optimized(text, pattern):
    if pattern == "":
        return list(range(len(text) + 1))
    lps = build_lps(pattern)
    hits = []
    i = j = 0
    while i < len(text):
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                hits.append(i - j)
                j = lps[j - 1]  # allow overlapping matches
        elif j:
            j = lps[j - 1]
        else:
            i += 1
    return hits
```

C4. Closest login optimisation with safe pruning

First, sort events by x-coordinate. For each event, examine later events in x-order. Once $(xj-xi)^2$ is at least the best squared distance already found, stop scanning that i: all later points have an equal or larger x separation, so none can be closer. This pruning is correct because Euclidean squared distance is dx^2+dy^2 and cannot be smaller than dx^2 alone. The algorithm avoids square roots and repeated/self pairs. It has O(n log n) sorting overhead and $O(n^2)$ worst-case time, but often prunes many comparisons in spatially spread data.

```
def closest_logins_optimized(events):
    if len(events) < 2:
        return None, None
    ordered = sorted(enumerate(events), key=lambda item: item[1][0])
    best2 = float("inf")
    best_pair = None
    for i in range(len(ordered)):
        original_i, (xi, yi) = ordered[i]
        for j in range(i + 1, len(ordered)):
            original_j, (xj, yj) = ordered[j]
            dx = xj - xi
            if dx * dx >= best2:
                break
            dy = yj - yi
            d2 = dx * dx + dy * dy
            if d2 < best2:
                best2 = d2
                best_pair = (original_i, original_j)
    return best_pair, math.sqrt(best2)
```

This is a pruned brute-force strategy, not a guaranteed O(n log n) closest-pair algorithm. In worst-case arrangements it may still examine $O(n^2)$ pairs. For a strict production latency guarantee at 8,000 events, a full divide-and-conquer closest-pair implementation or a spatial index should be evaluated.

Part (d) – Analyse Before and After (10 Marks)

D1. Complexity comparison

Function	Original / corrected baseline	Optimised approach	Main operation growth
Alert sort	Insertion Sort: best O(n), average/ worst $O(n^2)$	Stable Timsort: O(n log n) worst case; adaptive on nearly sorted data	Comparisons and element moves
Blocklist	Sequential search: O(b) per query	Set: expected O(1) per query after O(b) build	IP comparisons / hash membership
Signature scan	Correct brute force: O((n−m+1)m)	KMP: O(n+m)	Character comparisons
Closest logins	Unique-pair brute force: $n(n-1)/2 = O(n^2)$	X-sorted pruning: O(n log n) sort + $O(n^2)$ worst case, often fewer distance calculations	Squared-distance calculations

D2. Closest-pair distance calculations

The following reproducible experiment uses deterministic pseudo-random 2D coordinates in a 100,000 km × 100,000 km square (seed 20261111). The baseline checks every unique pair. The pruned version sorts by x and stops when the x-gap alone cannot improve the current best distance. The counts are exact for the generated datasets.

Events	Brute-force calculations	Pruned calculations	Reduction
1000	499500	814	99.84
4000	7998000	3598	99.96
8000	31996000	4825	99.98

The brute-force counts are 499,500 for 1,000 events; 7,998,000 for 4,000; and 31,996,000 for 8,000. The pruned counts depend on the coordinate distribution and seed. They are produced by the accompanying code and demonstrate the effect of the x-gap lower bound on these test inputs. They do not change the worst-case $O(n^2)$ bound.

D3. Insertion Sort comparisons: nearly sorted vs random

Dataset	Number of alerts	Insertion comparisons	Shifts / movements
Nearly sorted descending list with 50 adjacent swaps	5000	5048	50
Random permutation of the same alerts	5000	6235327	6230336

The nearly sorted data requires far fewer comparisons because only a small number of inversions must be repaired. The random permutation contains many more inversions, so insertion sort performs many shifts and comparisons. This supports the $O(n+I)$ explanation. Timsort should be benchmarked as the production alternative; actual elapsed times should be measured on the submission computer.

D4. Interpretation

- Operation counts are more reliable than a single timing result because timing is affected by hardware, interpreter, and background activity.
- Blocklist set lookup is especially valuable when many queries reuse the same blocklist.
- KMP is preferable for repeated signature scanning because it avoids restarting comparisons from the beginning after every mismatch.
- The closest-pair pruning reduces work on the tested spatial data but does not remove the quadratic worst case.
- The production decision should be based on both measured workload distribution and the required response-time target.

Part (e) – Deliver Production-Quality Code (8 Marks)

The consolidated module below includes docstrings, comments and a test harness for ordinary and boundary cases. The code can be saved as sentinelops.py and run with Python 3. No external packages are required.

```
"""SentinelOps SOC alert-triage utilities."""

import math


def sort_alerts(alerts):
    """Return alerts by descending severity; preserve arrival order for ties."""
    return sorted(alerts, key=lambda alert: -alert[1])


def make_blocklist_index(blocklist):
    """Build a reusable set for expected O(1) average membership tests."""
    return set(blocklist)


def is_blocked(blocked_ips, ip):
    """Return True when ip is present in the blocklist set."""
    return ip in blocked_ips


def build_lps(pattern):
    """Build the KMP longest-prefix-suffix table."""
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps


def find_signature(log, pattern):
    """Return every zero-based occurrence of pattern, including overlaps."""
    if pattern == "":
```

```
        return list(range(len(log) + 1))
    lps = build_lps(pattern)
    hits = []
    i = j = 0
    while i < len(log):
        if log[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                hits.append(i - j)
                j = lps[j - 1]
        elif j:
            j = lps[j - 1]
        else:
            i += 1
    return hits


def closest_logins(events):
    """Return (original-index pair, distance); (None, None) if fewer than 2."""
    if len(events) < 2:
        return None, None

    ordered = sorted(enumerate(events), key=lambda item: item[1][0])
    best2 = float("inf")
    best_pair = None

    for i in range(len(ordered)):
        original_i, (xi, yi) = ordered[i]
        for j in range(i + 1, len(ordered)):
            original_j, (xj, yj) = ordered[j]
            dx = xj - xi
            # All later x values are at least this far away.
            if dx * dx >= best2:
                break
            dy = yj - yi
            d2 = dx * dx + dy * dy
            if d2 < best2:
                best2 = d2
                best_pair = (original_i, original_j)

    return best_pair, math.sqrt(best2)


def _tests():
    """Run functional and boundary tests; raise AssertionError on failure."""
```

```
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    assert sort_alerts(alerts) == [
        ("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)
    ]
    assert sort_alerts([]) == []
    assert sort_alerts([("x", 5), ("y", 5)]) == [("x", 5), ("y", 5)]

    blocked = make_blocklist_index(
        ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    )
    assert is_blocked(blocked, "10.0.0.9") is True
    assert is_blocked(blocked, "8.8.8.8") is False
    assert is_blocked(make_blocklist_index([]), "8.8.8.8") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCXYZ", "XYZ") == [3]  # final legal position
    assert find_signature("ABC", "LONGER") == []
    assert find_signature("AAAA", "AA") == [0, 1, 2]  # overlaps
    assert find_signature("ABC", "") == [0, 1, 2, 3]

    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    pair, distance = closest_logins(events)
    assert set(pair) == {0, 5}
    assert abs(distance - math.sqrt(2)) < 1e-12
    assert closest_logins([]) == (None, None)
    assert closest_logins([(1, 2)]) == (None, None)
    assert closest_logins([(0, 0), (0, 0)])[1] == 0.0

    print("All SentinelOps tests passed.")


if __name__ == "__main__":
    _tests()
```

GitHub and submission checklist

- Save the final code as sentinelops.py and run it using Python 3.
- Confirm the terminal displays: All SentinelOps tests passed.
- Create a GitHub repository, add the Python file and this report if required, commit the files, and copy the repository URL into the submission portal.
- Do not invent a GitHub URL; paste the actual repository link after creating it.
- Add screenshots of your own execution and any locally measured timing results before submission.

Conclusion

The original SentinelOps module failed because of boundary-condition errors, incorrect Boolean returns, incomplete string-match validation and an incorrect closest-pair objective. The nine defects were corrected systematically and checked against the supplied test data. The optimised design uses stable sorting, a reusable set for blocklist membership, KMP for signature scanning and x-sorted distance pruning for closest login events. The analysis shows why each change improves correctness or reduces work, while also identifying limits: the pruned closest-pair method remains quadratic in the worst case. The final test harness checks normal cases and the required boundary conditions.

Source basis: the supplied four-page CSA06 CO2 assessment, especially the scenario and test data on pages 1–2 and the requirements for parts (a)–(e) on pages 2–3.